MERN / JavaScript — Units I, II & III

Exam-focused question bank: theory, definitions, comparisons and practical code examples.

For long answers, write definition + working + example/code + uses or limitations. This guide covers the main syllabus areas; exact exam questions cannot be guaranteed.

UNIT I — JavaScript Fundamentals

Introduction, features and ways to add JavaScript

Q. What is JavaScript?

Answer: JavaScript is a high-level, dynamically typed programming language used to add interactivity to web pages. It runs in browsers and in server-side runtimes such as Node.js.

Q. List the features of JavaScript.

Answer: Dynamic typing, first-class functions, prototype-based objects, event-driven programming, asynchronous operations, browser support, and modern module syntax.

Q. What are the ways to add JavaScript to HTML?

Answer: Inline: code in an event attribute; internal: code inside a script element; external: load a separate .js file. External scripts are generally easier to maintain.

```
<script>
  console.log("Hello");
</script>
<script src="app.js" defer></script>
```

Variables: var, let and const

Q. Differentiate var, let and const.

Answer: var is function-scoped and can be redeclared. let is block-scoped and can be reassigned but not redeclared in the same scope. const is block-scoped, must be initialized and cannot be reassigned; object properties can still be changed.

```
var oldValue = 10;
let score = 20;
const site = "Example";
score = 25; // valid
```

Data types and scope

Q. Explain JavaScript data types.

Answer: Primitive types: string, number, bigint, boolean, undefined, symbol and null. Non-primitive values include objects, arrays and functions.

```
let name = "Aman";
let age = 21;
let active = true;
let value;              // undefined
let empty = null;
let list = [1, 2, 3];
let user = {name: "Aman"};
```

Q. What is scope? Explain its types.

Answer: Scope determines where a variable can be accessed. Global scope applies broadly; function scope applies inside a function; block scope applies inside braces for let/const. Lexical scope allows inner functions to access variables in surrounding scopes.

Operators and conditional statements

Q. Explain JavaScript operators.

Answer: Arithmetic (+, -, *, /, %, **), assignment (=, +=), comparison (===, !==, >), logical (&&, ||, !), ternary (?:), typeof, nullish coalescing (??) and optional chaining (?.). Prefer === when type conversion is not wanted.

```
let marks = 78;
if (marks >= 90) console.log("A");
else if (marks >= 60) console.log("B");
else console.log("Needs improvement");

let day = 2;
switch (day) {
  case 1: console.log("Monday"); break;
```

```
  case 2: console.log("Tuesday"); break;
  default: console.log("Unknown");
}
```

Loops, comments and functions

Q. Explain for, while, do-while, for-of and for-in.

Answer: for is useful for counted iteration; while repeats while a condition is true; do-while executes at least once; for-of iterates iterable values; for-in iterates enumerable property keys, commonly on objects.

```
for (let i = 1; i <= 3; i++) console.log(i);
const colors = ["red", "blue"];
for (const color of colors) console.log(color);
const person = {name: "Riya", age: 20};
for (const key in person) console.log(key, person[key]);
```

Q. What are comments?

Answer: Comments document code and are ignored by the engine. // creates a single-line comment; /* ... */ creates a multi-line comment.

Q. Explain function declaration, expression, arrow and anonymous function.

Answer: A function is reusable code. A declaration uses function name() and is hoisted. An expression assigns a function to a variable. An anonymous function has no name. Arrow functions use => and capture this lexically.

```
function add(a, b) { return a + b; }
const multiply = function(a, b) { return a * b; };
const square = n => n * n;
console.log(add(2, 3), multiply(2, 3), square(4));
```

map, filter, reduce and event loop

Q. Explain map(), filter() and reduce().

Answer: map transforms each element and returns a new array; filter returns elements that satisfy a condition; reduce combines array values into one accumulated result.

```
const nums = [1, 2, 3, 4, 5];
console.log(nums.map(n => n * 2));
console.log(nums.filter(n => n % 2 === 0));
console.log(nums.reduce((sum, n) => sum + n, 0)); // 15
```

Q. Explain the event loop.

Answer: The call stack executes synchronous JavaScript. Runtime APIs handle asynchronous operations. Completed callbacks are queued; the event loop schedules them when the stack is clear. Promise reactions are microtasks and normally run before timer callbacks.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Output: A, D, C, B
```

Unit I possible questions

• Define JavaScript and list its features.

• Differentiate var, let and const.

• Explain primitive and non-primitive data types.

• Explain global, function, block and lexical scope.

• Explain operators and conditional statements with examples.

• Compare loop types.

• Explain functions and anonymous/arrow functions.

• Write code using map, filter and reduce.

• Explain the event loop with output order.

UNIT II — Arrays, Objects, ES6, DOM, Errors, Async JS and APIs

Arrays, objects and destructuring

Q. What is an array?

Answer: An array is an ordered collection of values with zero-based indexes. Common methods include push, pop, slice, splice, map, filter and reduce.

Q. What is an object?

Answer: An object stores data as key-value properties and may contain methods. Properties can be accessed with dot or bracket notation.

```
const numbers = [10, 20, 30];
const [first, second] = numbers;
const student = {name: "Neha", marks: 85};
const {name, marks} = student;
console.log(first, second, name, marks);
```

Q. What is destructuring?

Answer: Destructuring extracts array values or object properties into variables using concise syntax.

ES6 features

Q. List important ES6 features.

Answer: let/const, arrow functions, template literals, default parameters, destructuring, spread/rest, classes, modules, Promises and enhanced object literals.

```
const name = "Aman";
console.log(`Hello, ${name}`);
const old = [1, 2];
const copy = [...old, 3];
function sum(...values) {
  return values.reduce((a, b) => a + b, 0);
}
```

DOM manipulation

Q. What is DOM?

Answer: The Document Object Model represents an HTML page as a tree of objects. JavaScript can select, modify, create or remove elements and register event handlers.

```
// HTML: <p id="msg">Old</p><button id="btn">Change</button>
const msg = document.querySelector("#msg");
document.querySelector("#btn").addEventListener("click", () => {
  msg.textContent = "Updated";
  msg.style.color = "blue";
});
```

Q. Differentiate textContent and innerHTML.

Answer: textContent inserts or reads text. innerHTML parses HTML markup and can cause cross-site scripting if untrusted input is inserted. Prefer textContent for user-provided text.

Error handling

Q. Explain try, catch, finally and throw.

Answer: try contains risky code; catch handles an exception; finally runs whether or not an exception occurred; throw raises a custom error.

```
try {
  const data = JSON.parse('{"ok": true}');
  console.log(data.ok);
} catch (err) {
  console.error(err.message);
} finally {
  console.log("Finished");
}
```

Callbacks, Promises, async/await

Q. What is a callback?

Answer: A callback is a function passed to another function to be invoked later or when an operation completes. Deeply nested callbacks can become hard to maintain.

Q. What is a Promise?

Answer: A Promise represents an eventual asynchronous result. Its states are pending, fulfilled and rejected. then handles success, catch handles rejection, and finally runs after settlement.

```
const task = new Promise((resolve, reject) => {
  resolve("Done");
```

```
});
task.then(value => console.log(value))
    .catch(err => console.error(err))
    .finally(() => console.log("Complete"));
```

Q. Explain async and await.

Answer: An async function always returns a Promise. await pauses that async function until the awaited Promise settles, while the runtime can do other work. Use try/catch for failures.

```
async function loadUser() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users/1");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const user = await res.json();
    console.log(user.name);
  } catch (err) {
    console.error(err.message);
  }
}
loadUser();
```

Q. Differentiate callbacks, Promises and async/await.

Answer: Callbacks pass a function to run later; Promises represent a future result and allow chaining; async/await is syntax built on Promises that often makes asynchronous code easier to read.

Fetch, Axios and JSON

Q. What is Fetch API?

Answer: fetch() is a built-in API that returns a Promise for a Response. HTTP statuses such as 404 do not automatically reject the Promise, so check response.ok and then parse the body.

```
async function getPosts() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  if (!res.ok) throw new Error("HTTP " + res.status);
  const posts = await res.json();
  console.log(posts);
}
getPosts().catch(console.error);
```

Q. What is Axios? Compare Axios and Fetch.

Answer: Axios is a popular HTTP client installed as a package. Fetch is built into modern browsers. Axios generally parses JSON automatically and rejects non-2xx status codes by default; both require error handling.

```
// npm install axios
import axios from "axios";
async function getUsers() {
  try {
    const res = await axios.get("https://jsonplaceholder.typicode.com/users");
    console.log(res.data);
  } catch (err) { console.error(err.message); }
}
```

Q. Explain JSON.parse and JSON.stringify.

Answer: JSON.stringify converts a JavaScript value into JSON text. JSON.parse converts valid JSON text into a JavaScript value. JSON is a text format and does not support functions, undefined or comments.

```
const user = {name: "Aman", age: 21};
const text = JSON.stringify(user);
const restored = JSON.parse(text);
console.log(text, restored.name);
```

Unit II possible questions

• Explain arrays, objects and destructuring.

• Write notes on ES6 features.

• Explain DOM and event listeners.

• Create and append an element dynamically.

• Explain try/catch/finally and throw.

• Compare callbacks, Promises and async/await.

• Explain Fetch and Axios with GET examples.

• Explain JSON.parse and JSON.stringify.

• Write a program to fetch API data and display it.

UNIT III — React and Node.js

React: introduction, components, JSX, props and state

Q. What is React?

Answer: React is a JavaScript library for building user interfaces using reusable components and a declarative approach.

Q. Explain component, JSX, props and state.

Answer: A component is a reusable UI unit. JSX resembles HTML inside JavaScript. Props are read-only inputs passed by a parent. State is component-managed data that can change and trigger a re-render.

```
function Welcome({ name }) {
  return <h1>Hello, {name}</h1>;
}
// Usage: <Welcome name="Aman" />
```

Q. What is useState?

Answer: useState is a React Hook that returns the current state and a setter function. Calling the setter requests a re-render.

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>
    Count: {count}
  </button>;
}
export default Counter;
```

Q. Differentiate props and state.

Answer: Props are passed into a component and treated as read-only. State is managed by a component and changes over time.

Node.js and features

Q. What is Node.js?

Answer: Node.js is a JavaScript runtime built on the V8 engine that runs JavaScript outside the browser. It is used for servers, APIs, command-line tools and real-time applications.

Q. List features of Node.js.

Answer: Event-driven architecture, non-blocking I/O, large npm ecosystem, JavaScript on the server, and suitability for I/O-heavy systems. Long synchronous CPU work can block the event loop.

Q. Is Node.js a language or framework?

Answer: No. Node.js is a runtime environment for executing JavaScript. Frameworks such as Express run on Node.js.

NPM, setup and debugging

Q. What is NPM?

Answer: NPM is the Node Package Manager and package ecosystem. It installs dependencies and runs scripts defined in package.json.

```
npm init -y
npm install express
npm install --save-dev nodemon
node app.js
```

Q. What is package.json?

Answer: It contains project metadata, scripts, dependencies and configuration. package-lock.json records the resolved dependency tree for more reproducible installs.

Q. How do you debug a Node.js app?

Answer: Use stack traces, console logs, VS Code breakpoints and Node's inspector. Do not log passwords, tokens or sensitive user data.

```
node --inspect app.js
node --inspect-brk app.js
```

Node process model and event loop

Q. Explain the Node.js process model.

Answer: JavaScript usually runs on a main event-loop thread while many I/O operations are handled by OS facilities or a worker pool. When work completes, callbacks become eligible to run. Long synchronous work blocks other callbacks.

Q. Explain the Node.js event loop.

Answer: It coordinates asynchronous work across phases such as timers, pending callbacks, poll, check and close callbacks. Promise microtasks also run according to Node.js scheduling rules.

Modules: require, exports and import

Q. What is a module?

Answer: A module is a reusable unit of code. Node.js supports CommonJS and ECMAScript modules.

Q. Explain CommonJS require and module.exports.

Answer: require() loads a CommonJS module; module.exports exposes values to other files. exports initially refers to module.exports, so adding properties works, but reassigning exports alone does not replace module.exports.

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };

// app.js
const { add } = require("./math");
console.log(add(2, 3)); // 5
```

Q. Explain ES module import and export.

Answer: ES modules use export to expose values and import to use them. Enable them with an appropriate package.json type setting or .mjs files.

```
// math.js
export function add(a, b) { return a + b; }

// app.js in an ES-module project
import { add } from "./math.js";
console.log(add(2, 3));
```

Create a basic Node HTTP server

```
const http = require("http");
const server = http.createServer((req, res) => {
  res.writeHead(200, {"Content-Type": "text/plain"});
  res.end("Hello from Node.js");
});
server.listen(3000, () => console.log("Server on port 3000"));
```

Q. Differentiate React and Node.js.

Answer: React is a frontend UI library. Node.js is a runtime for executing JavaScript outside the browser, often for backend APIs and server logic.

Unit III possible questions

• Define React and explain components, JSX, props and state.

• Write a React counter using useState.

• Differentiate React and Node.js.

• Define Node.js and list features.

• Explain NPM and package.json.

• Explain Node process model and event loop.

• Explain setup and debugging.

• Explain CommonJS and ES module systems.

• Write a basic HTTP server.

Practical programs for revision

1. Largest of three numbers

```
const a = 12, b = 25, c = 18;
console.log(Math.max(a, b, c)); // 25
```

2. Even or odd

```
const n = 14;
console.log(n % 2 === 0 ? "Even" : "Odd");
```

3. Sum array with reduce

```
const values = [10, 20, 30];
console.log(values.reduce((sum, n) => sum + n, 0)); // 60
```

4. Filter passing marks

```
const marks = [32, 77, 49, 88, 51];
console.log(marks.filter(m => m >= 50)); // [77, 88, 51]
```

5. DOM button click

```
document.querySelector("#btn").addEventListener("click", () => {
  document.querySelector("#output").textContent = "Clicked!";
});
```

6. Fetch API data

```
async function main() {
  const res = await fetch("https://jsonplaceholder.typicode.com/users/1");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const data = await res.json();
  console.log(data.name);
}
main().catch(console.error);
```

7. Error handling for JSON

```
try {
  JSON.parse("{invalid}");
} catch (err) {
  console.log("Invalid JSON");
}
```

Quick revision checklist

• var function-scoped; let/const block-scoped.

• map transforms, filter selects, reduce accumulates.

• Promise states: pending, fulfilled, rejected.

• Fetch: check response.ok before parsing JSON.

• JSON.stringify: value to text; JSON.parse: text to value.

• DOM: querySelector, addEventListener, textContent.

• React: components, JSX, props, state, Hooks.

• Node: runtime; npm: packages/scripts; CommonJS: require/module.exports; ES modules: import/export.

• Prefer textContent over innerHTML for untrusted text.